



POLITECNICO
MILANO 1863

Drone Detection

Using Lightweight Feature-Enhanced CNN

Course: Numerical Analysis For Machine Learning (NAFML)

Louis-Aïtor Donval

Politecnico di Milano

June 1, 2026

Abstract

This report details the development and implementation of a lightweight convolutional neural network for the detection of small-scale drones at low altitudes. Developed "from scratch" as part of the Numerical Analysis for Machine Learning course, this project has the ambition to explore incremental complexity of architectures, starting from a basic toy model to a more sophisticated lightweight feature-enhanced network. We investigate the challenges of real-time detection in complex environments.

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Problem Statement: The Challenge of Small Objects	4
1.3	Project Scope and Academic Context	4
2	Reference Study Analysis	6
2.1	Overview of the Reference Paper	6
2.2	Architectural Pillars: LSM, EFM, and ADM Components	6
2.2.1	Lightweight and stable feature extraction module (LSM)	6
2.2.2	Feature Extraction Module (EFM)	7
2.2.3	Accurate Detection Module (ADM)	10
2.3	The Loss Function : From MSE to CIoU	11
2.4	Theoretical Challenges	12
2.4.1	The Anchor Mechanism and K-means Clustering	12
2.4.2	Balancing Spatial vs. Semantic Information	12
3	Implementation & Methodology	13
3.1	Dataset Acquisition and Processing	13
3.1.1	Data Sources	13
3.1.2	Dataset Samples	13
3.1.3	Data Splitting Strategy	14
3.2	Data Augmentation Strategy	14
3.2.1	Technical Implementation with Albumentations	14
3.3	Incremental Model Development	15
3.3.1	Model V0: The Baseline Toy Model	15
3.3.2	Model V1: Loss Function Optimization	15
3.3.3	Model L1: Lightweight Search Module	16
3.3.4	Model LA1: LSM + Attention detection module (ADM)	16
3.3.5	Model LAC1: Final Architecture	16
3.4	Training Environment and Optimization Strategy	16
3.4.1	Hardware and Software Specifications	17
3.4.2	Hyperparameters	17
3.4.3	Numerical Optimizer: The Adam Algorithm	17
3.4.4	Activation function : Leaky ReLU	18

4	Results and Performance Evaluation	19
4.1	Comparative Metric Analysis	19
4.1.1	Comprehensive Performance Table	19
4.1.2	The Sensitivity/Precision Trade-off	20
4.2	Qualitative Results and Detection Samples	20
4.2.1	The Impact of the Loss Function	20
4.2.2	Different tests	21
5	Discussion, Limits, and Conclusion	23
5.1	Error Analysis	23
5.1.1	Increasing architecture complexity and architectural simplifications	23
5.1.2	Miss constructed dataset	23
5.2	Future Work and Suggestions for Improvement	24
5.3	Conclusion	24
6	Appendix : Usage of AI	25

1 Introduction

1.1 Background and Motivation

In recent years, the rapid advancement and commercial availability of Unmanned Aerial Vehicles (UAVs), known as drones, had a major impact. This proliferation has introduced significant challenges regarding public safety, privacy, and airspace security. The ability of drones to operate at low altitudes makes them difficult to track with traditional radar systems, creating a critical need for robust, vision-based automated detection systems. The misuse or accidental presence of a drone in restricted areas poses a growing threat. Efficiently detecting these objects is no longer just a technical curiosity but a fundamental requirement for modern security of infrastructure and people.

1.2 Problem Statement: The Challenge of Small Objects

Detecting drones in real-time is a non-trivial task in computer vision. Unlike standard object detection (such as cars or pedestrians), drones often appear as "small-sized objects" in images, occupying only a tiny fraction of the total pixels. This leads to several technical challenges:

- **Loss of Spatial Information:** Consecutive downsampling in traditional Convolutional Neural Networks (CNNs) often erases the fine features of small objects.
- **Complex Backgrounds:** Drones are frequently obscured by birds, clouds, or complex urban landscapes, leading to high false-positive rates.
- **Resource Constraints:** For security applications, detection must often happen on on-site cameras or mobile units with limited computational power, requiring lightweight and highly optimized architectures.

1.3 Project Scope and Academic Context

This project was developed as part of the *Numerical Analysis For Machine Learning* (NAFML) course at **Politecnico di Milano**. The primary objective of this work is to bridge the gap between the theoretical numerical foundations coming from the course, and a complex real-world application. Inspired by the study "*Low-altitude small-sized object detection using lightweight feature-enhanced convolutional neural network*¹", I aimed to reproduce their findings on a smaller scale.

¹Y.Tao¹, Z. Zongyang, Z. Jun, C. Xinghua, and Z. Fuqiang, Journal of Systems Engineering and Electronics Vol. 32, No. 4, August 2021, pp.841 – 853

My choice was to develop the neural network architectures from scratch rather than using pre-trained weights or high-level wrappers like YOLOv8's default APIs. By building the models incrementally—starting from a basic "Toy Model" and moving towards more sophisticated architectures incorporating Lightweight Search Modules (LSM) and Accurate Detection Modules (ADM)—this project serves as a comprehensive exploration of how architectural choices impact detection performance.

2 Reference Study Analysis

2.1 Overview of the Reference Paper

The foundation of this project is the study "*Low-altitude small-sized object detection using lightweight feature-enhanced convolutional neural network*". The authors address a specific gap in the current state-of-the-art: YOLO, SSD models are highly effective for general object detection, but they often struggle with drones in low-altitude flight due to their small relative size and the high computational cost required for high-resolution feature extraction.

The reference paper introduces a specialized CNN architecture designed to be both computationally efficient and spatially sensitive. The use of optimized convolution blocks reduce the number of parameters and increase detection while being computationally light.

In the following sections, we analyze how the paper proposes to structure the data flow through four distinct modules: the **LSM** (Lightweight and stable feature extraction module) for dimension reduction and enhancing of low level feature, the **EFM** (Feature extraction module), and the **ADM** (Accurate detection module) identifies a scale for detection after feature information as been extracted by **LNB** (Lightweigh Network block).

By analyzing this paper, the objective of this project was to extract these complex modules and re-implement some of them incrementally.

2.2 Architectural Pillars: LSM, EFM, and ADM Components

The architecture proposed in the reference study introduce modular blocks designed to maximize feature extraction efficiency.

2.2.1 Lightweight and stable feature extraction module (LSM)

The LSM is responsible for primary feature extraction. In the context of small object detection, traditional convolutions often lead to a gap where high-level features lose the spatial precision needed. Moreover, the LSM reduce computational complexity while improving the feature-representation capability of the system.

The LSM addresses this by using a multi-pathway approach that is presented on the following figure :

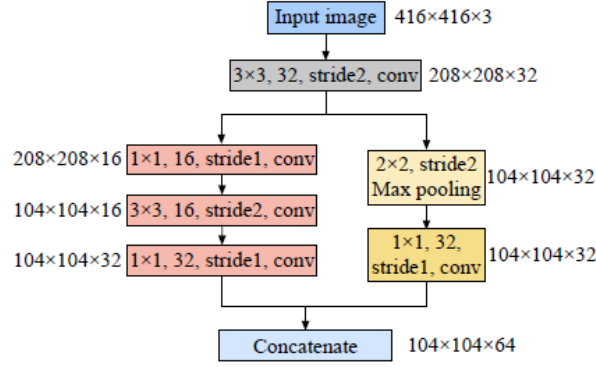


Figure 1: LSM

In my implementation, I adapted this by reducing the dimension of the LSM to ensure the model remained trainable while retaining the structural benefit of the search module.

2.2.2 Feature Extraction Module (EFM)

To further optimize the detection performance, the study integrates CSP structures inspire by CSPNet through the EFM :

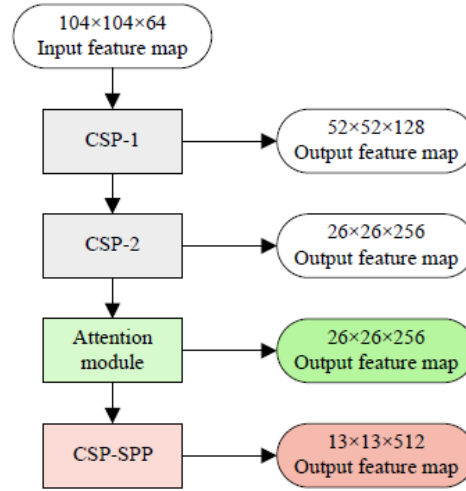


Fig. 2 EFM

Figure 2: EFM

Each CSP block achieves adequate feature extraction and reduces the size of the feature map by max-pooling :

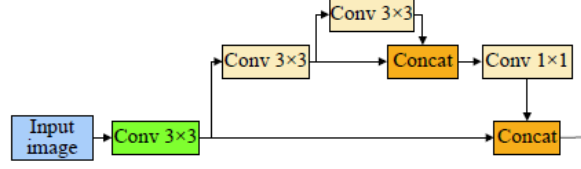


Fig. 3 Frame of CSP-1 and CSP-2

Figure 3: CSP

In my project, the addition of the first CSP made me reach over 70,000 parameters so I couldn't implement the whole structure while being able to train the model.

The attention module following is designed to increase the weight of significant information and reduce that of unimportant information to improve the detection ability :

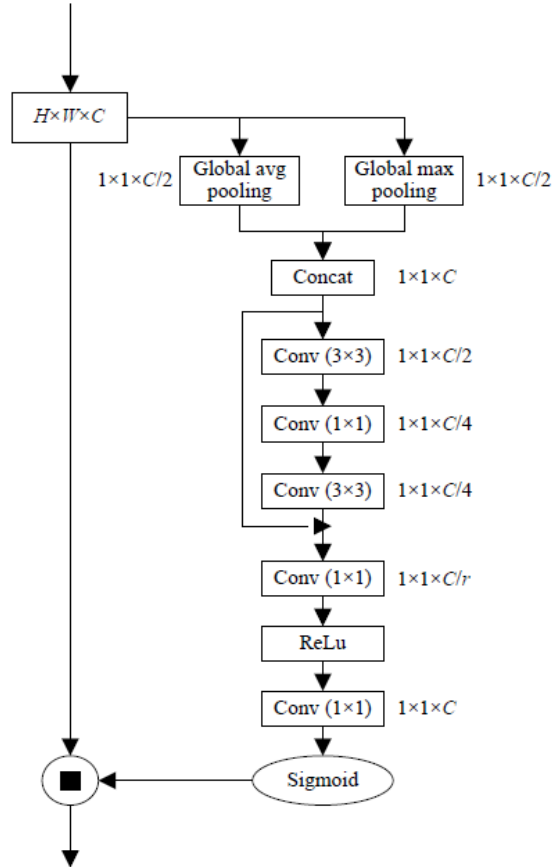


Figure 4: Attention module

Finally the SCP-SPP module enhance the feature extraction ability of the model :

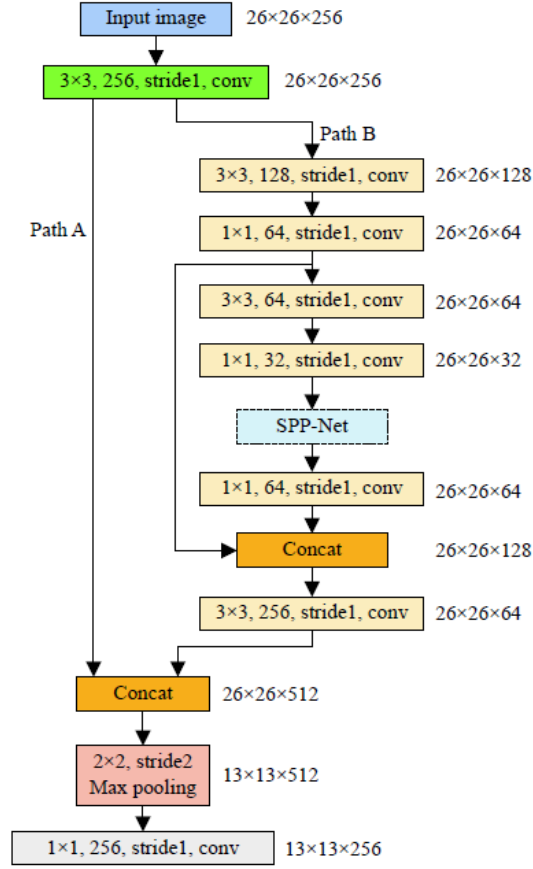


Figure 5: CSP-SPP Module

2.2.3 Accurate Detection Module (ADM)

The most complex parts of the reference study is the ADM. The study uses K-means to redefine the size of anchors. Then, the module determines which scale is privileged for an object based on his size, ensuring better detection performance as described below :

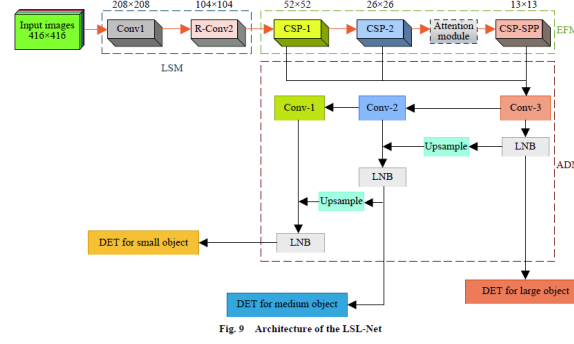


Figure 6: Entire architecture

We notice that a lightweight network block (LNB) is presented before the final detection at each scale. He's defined as follows :

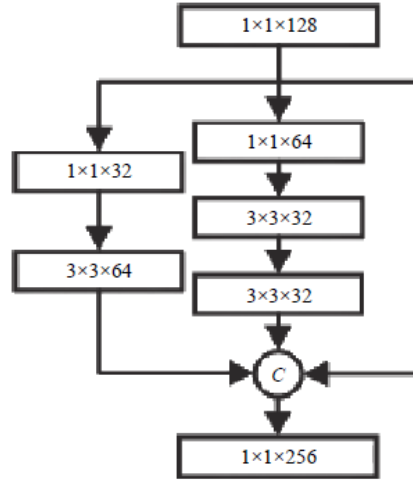


Figure 7: LNB module

It effectively improves the detection performance of the model with only a relatively insignificant increase in the amount of computation.

2.3 The Loss Function : From MSE to CIoU

The choice of a loss function is a really critical numerical decision in training a detection model. During the initial phase of this project, I will use a standard Mean Squared Error (MSE) as a baseline. However, empirical results soon highlighted the limitations of MSE for bounding box regression. MSE is convex and easily differentiable but, when applied to bounding box coordinates and labels, MSE treats each variable independently which leads to geometric ignorance and unbalanced tradeoff between confidence and geometric precision. The study presents instead a more complex loss function based on Complete Intersection over Union.

CIoU is a geometric loss that considers the physical properties of the boxes :

$$\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha\beta \quad (1)$$

Where:

1. IoU is a measurement of the overlapping of the boxes.
2. ρ is the Euclidean distance between the central points of the predicted box (b) and the ground truth box (b^{gt})
3. c is the diagonal length of the smallest enclosing box covering both.
4. $\alpha\beta$ measures the consistency of the aspect ratio.

For the label a cross entropy loss summed over the grid and the bounding boxes has been used : Where :

$$\begin{aligned} \text{loss}_1 = & - \sum_{i=0}^{S^2} \sum_{j=0}^B I_{ij}^{obj} [C_i^j \log(\widehat{C}_i^j) + (1 - \widehat{C}_i^j) \log(1 - C_i^j)] - \\ & \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B I_{ij}^{obj} [C_i^j \log(\widehat{C}_i^j) + (1 - \widehat{C}_i^j) \log(1 - C_i^j)] \end{aligned}$$

1. S^2 is the value of the input image grid numbers.
2. B the number of bounding boxes in a grid.
3. C_i^j are the confidence scores predicted and ground truth.
4. I_{ij}^{obj} indicates if an objetc is present or not in the corresponding bounding boxe and grid cell.

5. λ_{noobj} is a weighting parameter that define how much we penalize false positive.

The study also add a specific Loss on prediction probability on bounding boxes within a grid cell but I implemented the model with only one bounding box by grid cell for simplicity matters.

2.4 Theoretical Challenges

The implementation of the reference study raised several complex theoretical questions, particularly regarding the optimization of bounding box priors and the management of multi-scale information. This section provides a critical analysis of these mechanisms.

2.4.1 The Anchor Mechanism and K-means Clustering

Firstly, in the reference study they use K-means clustering to determine "anchors." That are pre-defined bounding boxes that serve as templates. The network does not predict the box from scratch; instead, it predicts the refinements pre computed anchors. It has several advantages :

- By clustering all ground-truth boxes the K-means algorithm identifies the most frequent drone shapes and sizes.
- If the initial anchors are close to the real shapes of the drones, the optimization is much shorter.

In my project I didn't really understood where the K-means clustering results are shared to the detection algorithm so I wasn't able to implement this specific part.

2.4.2 Balancing Spatial vs. Semantic Information

Another challenge lies in how the architecture prioritizes different object sizes. In deep learning, there is a natural trade-off between:

- **Spatial Information:** Located in the first layers, there are features with high resolution but low semantic meaning.
- **Semantic Information:** Located in deeper layers, these features understand more the object but have low resolution due to successive pooling.

The reference study balances this through a Multi-Scale Detection strategy. Depending on object size, the detection process use different depths and anchors size. The smaller the objects are the deeper the layers and the smaller the anchor sizes will be.

During my project, I have not been able to understand how the process detect which of the scale should be used and I wasn't able to implement it.

3 Implementation & Methodology

3.1 Dataset Acquisition and Processing

The success of any deep learning project, relies heavily on the quality of the dataset. For this project, I choose to diversify the sources of training and testing processes to provide both a robust training and realistic testing scenarios.

3.1.1 Data Sources

Two sources were utilized to build the project:

- **Primary Image Dataset:** Dataset compiled by *Mehdi Özel*, available via GitHub². This dataset consists of 1,359 high-quality images of drones. These images are particularly valuable as they feature various lighting, angles, and backgrounds, and are labelled.
- **Video Test Bench:** Then to evaluate the models in real-world conditions, I found royalty-free videos of from *Vecteezy*³. These videos include drones but also seagulls, waves, and clouds, which are essential for testing the model’s robustness.

3.1.2 Dataset Samples

To illustrate the variety and scale of the objects to be detected, I show some samples from the Mehdi Özel dataset.



Image 0003

Image 0004

Image 0005

Image 0006

Figure 8: Samples from the training dataset showing different drone scales and environments.

²<https://github.com/dasmehdix/drone-dataset>

³<https://www.vecteezy.com/>

3.1.3 Data Splitting Strategy

A 60/20/20 split was implemented to ensure the integrity of the numerical validation process, following standard machine learning practices:

1. **Training Set (60%):** Used to optimize the weights of the different model versions using the Adam optimizer.
2. **Validation Set 1 - Threshold Optimization (20%):** This set was specifically used to calibrate the classification threshold. Given our objective to insure a minimum sensitivity, we fixed the threshold on this set to ensure a minimum sensitivity of 50%.
3. **Validation Set 2 - Model Comparison (20%):** A final set was used to compare the performance metrics (CIoU, Precision, Sensitivity, Inference speed) across all successive model versions.

This triple split approach seems necessary to me because, small variations in threshold can lead to significant shifts in the precision-recall curve.

3.2 Data Augmentation Strategy

Given the relatively small size of the original dataset (1359 images) and the complexity of the detection task, I used data augmentation. The objective was to artificially increase the diversity of the training set to prevent overfitting and improve the model's robustness to varying environmental conditions. In practice it works well to avoid overfitting but didn't improve much model performance.

3.2.1 Technical Implementation with Albumentations

We utilized the *Albumentations* library, which is specifically optimized for computer vision tasks involving bounding boxes. It simultaneously transform the image pixels and the corresponding label coordinates as we can see in below :

The choice of transformations was guided by the physical reality of drone flight:

- **Horizontal Flips, Shift, Scale, and Rotations:** Drones flying in different directions at different distances.
- **Random Brightness and Contrast:** Simulate different times of day and weather conditions.
- **Gaussian Noise, Dropout of pixels:** To mimic sensors issues noise often found in real-time video feeds.

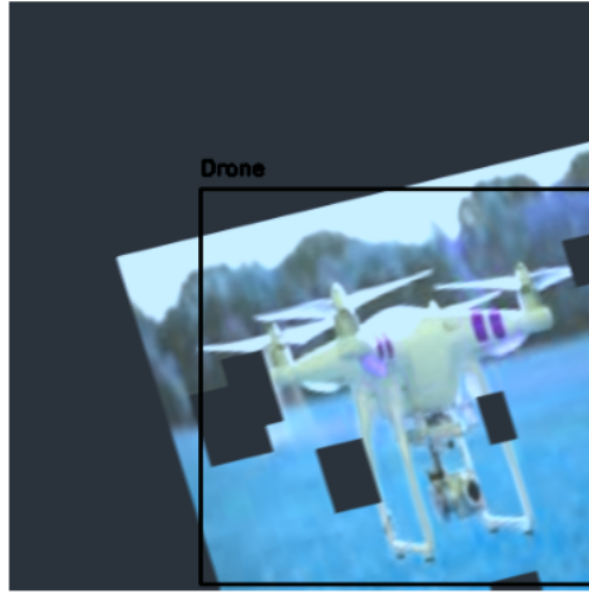


Figure 9: Augmented image with its updated bounding box

3.3 Incremental Model Development

I present briefly in this part the different models incrementally more complex that I have implemented and that we will explore.

3.3.1 Model V0: The Baseline Toy Model

The first model, `model_v0`, served as a minimalist baseline to establish a performance floor.

- **Architecture:** A sequential backbone of four 3×3 convolutional layers with Max-Pooling.
- **Loss Function:** A simple Mean Squared Error (MSE) applied uniformly across all outputs..

3.3.2 Model V1: Loss Function Optimization

Model V1 introduced a critical shift without changing the number of parameters.

- **Transition to YOLO-style Loss:** We replaced MSE with loss described earlier using Binary Cross-Entropy (BCE) for the confidence score and specific CIoU loss for coordinate regression.

3.3.3 Model L1: Lightweight Search Module

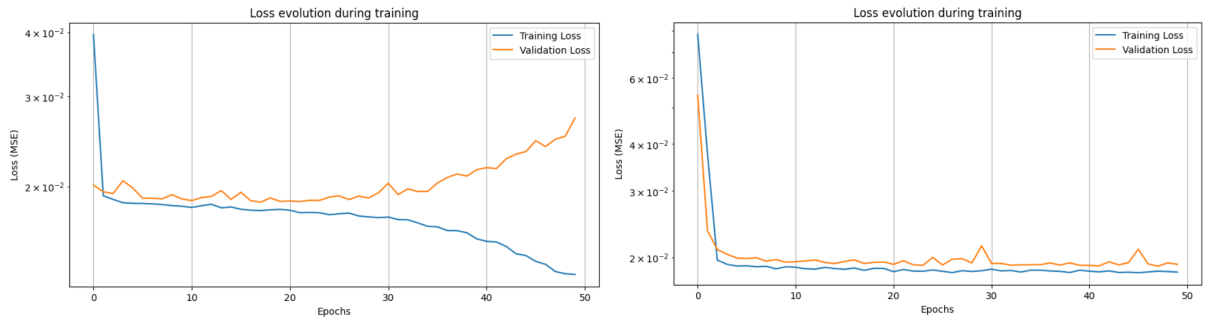
In Model L1 (3,939 parameters), we implemented the **LSM** pillar from the reference study.

- **Architecture:** We add the LSM module from the reference study.

3.3.4 Model LA1: LSM + Attention detection module (ADM)

Model LA1 (38,843 parameters) integrates the **ADM**.

- **Architecture:** We combine the LSM module and the ADM from the reference study.
- **The Overfitting Challenge:** This is where the augmentation started to be useful to avoid overfitting as shown below.



Overfitting before image augmentation

Regularization after image augmentation

Figure 10: Solving overfitting with image augmentation.

3.3.5 Model LAC1: Final Architecture

The final model, `model_lac1` (70,103 parameters), represents the more complex model, containing the three pillars of the reference study.

- **Architecture:** The model contain the three LSM + CSP + ADM

3.4 Training Environment and Optimization Strategy

The training of neural networks is a computationally intensive. Given the lightweight objective of this project, the entire development and training pipeline was executed on a standard laptop, simulating the constraints of edge-computing development.

3.4.1 Hardware and Software Specifications

The numerical experiments were conducted under the following environment:

- **Processor (CPU):** Intel Core i5-8250U @ 1.60GHz.
- **Software Stack:** Python 3 with *TensorFlow/Keras* for model orchestration.
- **Visualization:** Matplotlib and OpenCV for real-time detection rendering and video processing.

3.4.2 Hyperparameters

The training hyperparameters were defined as follows :

- **Epochs:** 50 (standard).
- **Batch Size:** 64 (I needed to chose a relatively high batch size to keep the training time acceptable).
- **Input Resolution:** 416×416 pixels, normalized between 0 and 1 (Reference study).

3.4.3 Numerical Optimizer: The Adam Algorithm

As explored in the NAML course, the choice of optimizer is critical. We selected the **Adam** (Adaptive Moment Estimation) optimizer due to its efficiency.

Adam combines the advantages of two other extensions of stochastic gradient descent: *AdaGrad* and *RMSProp*. It maintains an exponentially decaying average of past gradients (m_t) and past squared gradients (v_t):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad , \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (2)$$

Where g_t is the gradient at time t , and $\beta_1 = 0.9, \beta_2 = 0.999$ are the moving average coefficients. The parameter update rule is then:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (3)$$

Where $\eta = 10^{-3}$ is the learning rate, $\epsilon = 10^{-7}$ a constant to prevent division by zero, and $\hat{m}_t, \hat{v}_t$ are bias-corrected estimates.

The values of the parameters are the ones advised by the keras optimizer based on the *Kingma et al., 2014* study.

3.4.4 Activation function : Leaky ReLU

For the hidden layers of the network, Leaky ReLU (Rectified Linear Unit) was selected as the activation function. This choice was made to address the dying ReLU problem. In fact standard ReLU neurons can become permanently inactive if they output zero during training. By introducing a small, non-zero gradient for negative inputs, Leaky ReLU ensures that backpropagation can still update the weights. This leads to faster convergence and more stable training.

$$f(x) = \begin{cases} x & \text{si } x \geq 0 \\ \alpha x & \text{si } x < 0 \end{cases}$$

In our cases we fix $\alpha = 0.3$ which is what can be advised in the keyras documentation.

4 Results and Performance Evaluation

This section presents quantitative and qualitative evaluation of the developed models.

4.1 Comparative Metric Analysis

To evaluate our incremental approach, we measured key indicators: the number of parameters, training time, Average CIoU, Sensitivity and Prediction Speed.

4.1.1 Comprehensive Performance Table

The table below summarizes the performance of each model version. All metrics were recorded on the second validation set.

Table 1: Comparative performance metrics across incremental model versions.

Model	Parameters	Training Time	Speed	CIoU	Precision	Sensitivity
V0 (MSE loss)	1,309	330s	0.10s/pred	1.453	5.53%	50.68%
V1 (YOLO loss)	1,309	350s	0.10s/pred	0.623	17.00%	57.82%
L1 (LSM)	3,939	1,975s	0.11s/pred	0.510	25.61%	53.74%
LA1 (ADM)	38,843	12,302s	0.11s/pred	0.541	34.19%	53.06%
LAC1 (CSP)	70,103	18,130s	0.12s/pred	0.609	33.62%	65.31%

First of all, all the models reach an acceptable inference speed even with a large number of parameters. Then, we can see that we don't improve coordinate precision after the third model. However we reach better precision with the last two models. Finally the last model was better in term of sensitivity because it was the only one where lowering his sensitivty didn't really improve the precision so I choose to keep a higher sensitivity rate.

What is surprising is that the quality of the model is not always clearly improved by the number of parameters and the complexity of the architecture.

4.1.2 The Sensitivity/Precision Trade-off

A critical aspect of this project was the sensitivity constraint. In security matters, **Sensitivity** is often prioritized over Precision because it is safer.

The 50% Sensitivity Constraint: For each model, we dynamically adjusted the threshold on Validation Set 1 to ensure that at least **50% of the drones were detected**.

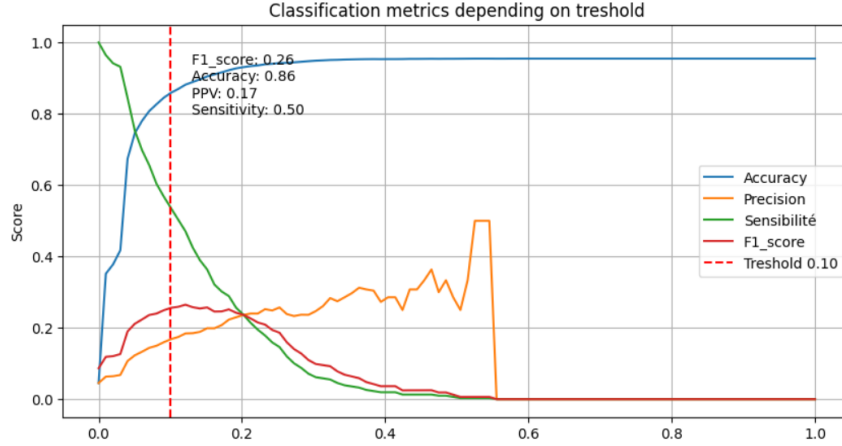


Figure 11: Example of analysis of the classification threshold optimization to reach the 50% sensitivity target.

For all model except the last one reducing sensitivity increase the precision, so the lower bound for the sensitivity is meaningful. Nevertheless, for the Model LA1 it was not clearly the case, so the threshold has been set to the maximum value of the sensitivity that insure same precision as the 50% sensitivity lower bound.

4.2 Qualitative Results and Detection Samples

While numerical metrics provide an objective overview, a qualitative visual analysis is essential to understand the specific failures and successes of our models.

4.2.1 The Impact of the Loss Function

The transition of the loss function is actually the most dramatic improvement in this project. The first model with MSE loss is the only one with test results significantly different from the others.



Figure 12: Baseline V0 (MSE): Bounding boxes covering irrelevant areas.

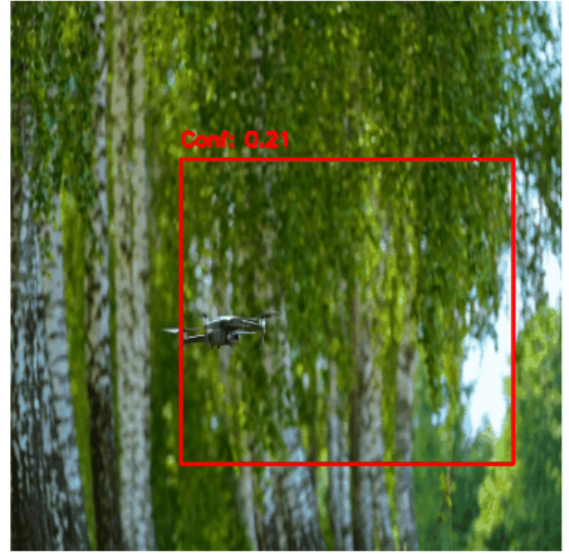


Figure 13: Model LAC1: Improvement in localization and confidence.

It is important to notice that this results are screenshots from a video test. Actually most of the time *Model V0* is not able to predict anything, and even with the more advanced model, we lost the prediction at least one over two frames (we are working at 8FPS).

4.2.2 Different tests

We tested the last model with different test videos.

Sky background : After the nature background we tested our model on sky background which would be a standard case during exploitation of drone detector. Surprisingly even if the background is pretty simple our model often fail to detect the drone.

Videos without drones : We also wanted to know if the model behavior in drone free context to evaluate its precision. A video of seagulls and another of the sea have been used. The results are not good at all. The model is clearly unable to discriminate seagulls and drones and moreover even in the sea context the model sometimes detects drones.



Figure 14: Sky background frame example of fail detection

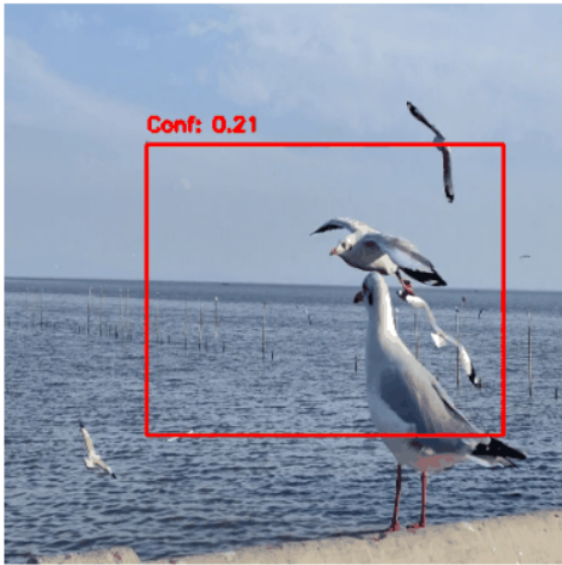


Figure 15: Seagulls missclassified as drone.



Figure 16: Sea context.

Moreover it's is clear that the model only detect object centered in the image, in most of the frames if a drone is detected the predicted coordinates are clearly biased towards the center of the image which is an issue.

We can conclude that we are not satisfied at all with the result of the test, and that our models have shown disapointing perfomances.

5 Discussion, Limits, and Conclusion

5.1 Error Analysis

Having to rebuild from scratch make implementation really interesting but can also easily lead to mistakes and lack of perspective due to the multiplication of challenges. We'll explore in this part what were the main problems of the project design.

5.1.1 Increasing architecture complexity and architectural simplifications

Dimension reduction : An interesting numerical observation from our results is that `model_lac1`, despite its higher complexity and the integration of CSP blocks, did not significantly outperform `model_la1` in terms of pure precision. More, as said before it has not been clear that increasing complexity has always improve model performance. Having tackle overfitting, the main problem is that we had to reduce a lot the dimension of the convolution used in the block added to keep the model trainable. These simplifications have been made uniformly without much consideration on what where the critical part of the architecture. It is very likely that we have reduced in a dramatic way some of the key parts of the pillars blocks making the model unable to retrieve informations initially aimed.

No multiscale detection : As explained before we only work on the first scale detector which is designed for the big objects. In particular, this can explained why it seems that even with small object the predictive bounding box is always really large. More important, this lead to a lack of generalization for different scale objects.

No K-means approach : Having implemented the K-means approach could have also helped us to have more robust bounding boxes and better convergence during training.

5.1.2 Miss constructed dataset

No empty sample I thought at first sight that the grid structure creating multiple empty cells would have be enough for the model to be able to detect the absence of the drone. Nevethless, the model has shown during the tests that it is not the case and that even waves or cloud can be confusing.

No labelisation of false friends : Drones have typical false friends such as birds and planes. I should have tried to also classified this critical objects that are by essence very confusing for the models. First of all, I should have at least include these objects in the

dataset. Actually, in the study they also classified planes and birds to help the model detect these confusing objects. I didn't implemented it for simplicity matter thinking it was a refinement but I think that it was actually a really important feature.

5.2 Future Work and Suggestions for Improvement

The limitations identified in the previous sections provide a clear roadmap for future improvements of the project :

- **Implement more of the reference study features :** Adding the Multiscale detection and using the K-means approach can lead to significantly better results.
- **Tune the dimension reduction of the layers :** Ideally with a more powerful computer unit we could replicate the entire architecture. However in our case we should incrementally and very carefully reduce the dimension of the different building blocks to understand the ones that are critical and need to be prioritize.
- **Enriching the Dataset and classify false friends :** Include in the dataset significant proportion of images containing birds, planes and clouds and add their classification in the model.

5.3 Conclusion

Despite the disappointing results, this project achieved its goal to link the theoretical foundations of computer vision discovered within the *Numerical Analysis for Machine Learning* course and the practical complexities of a real world computer vision project. My objective was to move beyond the black-box approach often associated with deep learning, and increasing my understanding of the numerical mechanics, hardware limitations and other challenges of machine learning applications.

This project allowed me to deep dive into real world application and its intransigence. I've cover lot of very interesting topics from objective function choice and hardware usage optimization to dataset construction and neural network architecture design.

Even if the models hasn't reach the performance I expected, this project was a great journey and has motivated me to continue studying computer vision.

6 Appendix : Usage of AI

LLMs have been used during this project for the following tasks :

- To develop vizualization tools in Python
- To develop format conversion tools in Python
- To create example of method usages in Python
- To produce the report (LateX syntax and architecture, english translation).

References

- [1] Albumentations Team. Bounding boxes augmentations for object detection. Albumentations Documentation.
- [2] Edie Miglio. Numerical analysis for machine learning. Course notes, Politecnico di Milano, 2025. Academic Year 2025–2026.
- [3] Mehdi Özel. Drone dataset. GitHub repository, 2021.
- [4] Y. Tao, Z. Zongyang, Z. Jun, C. Xinghua, and Z. Fuqiang. Low-altitude small-sized object detection using lightweight feature-enhanced convolutional neural network. *Journal of Systems Engineering and Electronics*, 32(4):841–853, August 2021.
- [5] TensorFlow Authors. Tensorflow tutorials and keras usage guides. TensorFlow Documentation.
- [6] Vecteezy. Royalty-free stock videos. Online Video Platform.